Project - Low Level Design

On

Multi-Agent Insurance System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1	S. JEZREEL	EN22IT301085
2	KRITAGYA TIWARI	EN22IT301048
3	SADHANA YADAV	EN24CA5030146
4	CHANDRESH BHATI	EN24CA50040
5	SHIVKUMAR SOLANKI	EN24CA5030157

Project Number:DO-40

Industry Mentor Name: Mr. Yash

University Mentor Name: Prof. Ajeet Singh Rajput

Academic Year:2025-26

Group Name: 02D12

Table of Content

S.No	Title	Page No
1	Introduction	3
1.1	Scope of the Document	3
1.2	Intended Audience	4
1.3	System Overview	4
1.4	System Architecture Diagram	5
2	System Design	4
2.1	Application Design	6
2.1.1	Class Diagram	-
2.2	Process Flow	6
2.3	Information Flow	6
2.4	Components Design	7
2.5	Key Design Considerations	8
2.6	API Catalogue	9–11
3	References	-

1. INTRODUCTION:

The rapid development of Generative AI has enabled intelligent systems capable of performing complex tasks through collaboration between specialized agents. This project focuses on designing a **collaborative multi-agent system for insurance information generation**, where different AI agents work together to produce structured and meaningful outputs.

The system consists of two primary agents: a **Researcher Agent** responsible for gathering insurance-related information such as policy coverage, benefits, and conditions, and a **Writer Agent** responsible for organizing, synthesizing, and formatting this information into structured outputs. The agents communicate through defined protocols and share contextual information using state management techniques.

The system is implemented using **Python**, with frameworks such as **LangChain or CrewAI** to orchestrate agent collaboration and workflow management. A **Large Language Model (LLM) API** is used to process natural language queries and generate context-aware responses. The architecture demonstrates advanced AI design concepts such as **inter-agent communication, workflow orchestration, and modular agent specialization**.

1.1 Scope of the Document

This document provides a detailed **Low Level Design (LLD)** of the collaborative insurance agent system. It describes the internal architecture, components, workflows, and interactions between agents that enable the system to function effectively.

The document focuses on explaining how user queries are processed by specialized agents, how information flows between components, and how the final output is generated. It also provides details about system APIs, design considerations, and communication protocols used for agent collaboration. The purpose of this document is to offer a clear understanding of the system's implementation and technical structure.

1.2 Intended Audience

This document is intended for **software developers, AI engineers, project mentors, and system reviewers** who need to understand the internal design of the system. It provides detailed insights into the architecture, components, and workflow of the collaborative AI agent system.

The document can also be useful for students and researchers interested in learning about **multi-agent AI systems, workflow orchestration using LangChain or CrewAI, and the practical integration of LLM APIs** for intelligent applications.

1.3 System Overview

The system is designed as a **multi-agent AI application** that processes insurance-related queries and generates structured responses through collaboration between specialized agents. The application receives user queries related to insurance topics such as policy coverage, benefits, exclusions, and risk factors.

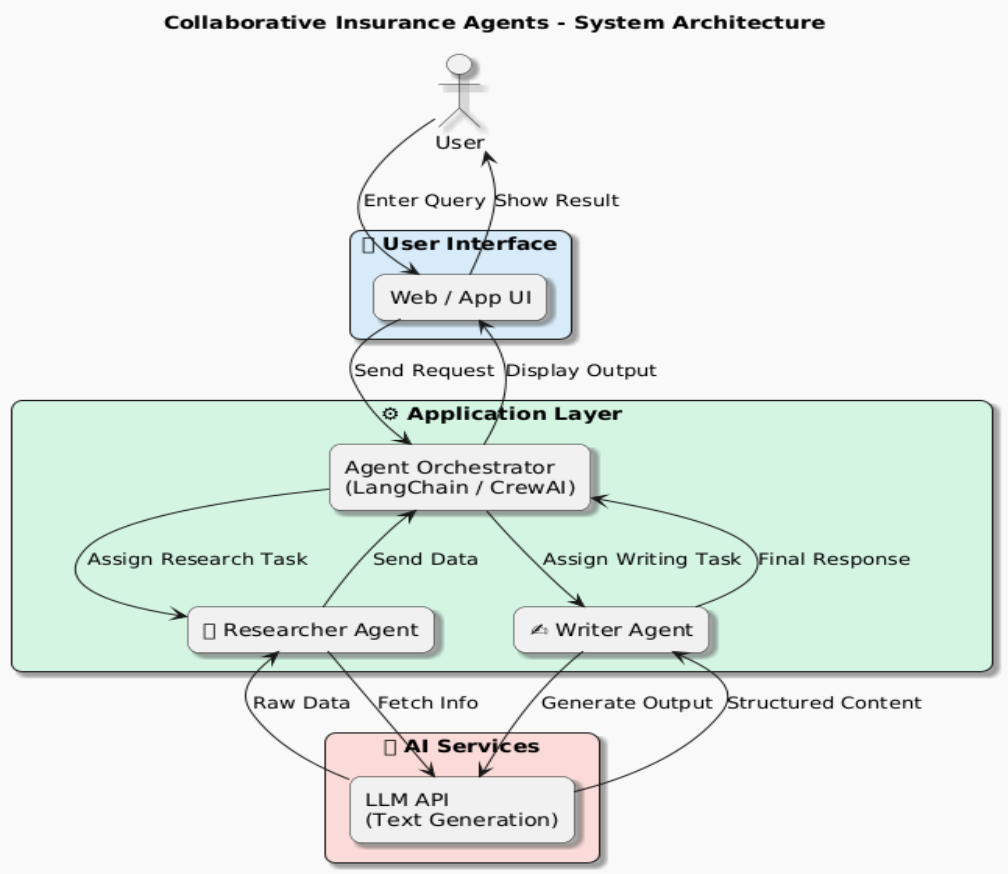
The query is first processed by the **Researcher Agent**, which retrieves and analyzes relevant information using LLM capabilities. The gathered information is then transferred to the **Writer Agent**, which refines the data and converts it into a structured and well-formatted response suitable for presentation or documentation.

The coordination between agents is managed using **LangChain or CrewAI**, which enables task assignment, communication, and workflow orchestration. The system demonstrates the practical implementation of **collaborative AI agents working together to solve complex tasks efficiently**.

1.4 System Architecture Diagram :

The system architecture of the Collaborative Insurance Agents application is designed using a multi-agent approach, where different agents collaborate to process and generate structured insurance-related outputs. The system consists of a central Agent Orchestrator, implemented using LangChain or CrewAI, which manages the workflow and communication between agents. When a user submits a query, the orchestrator assigns the task to the

Researcher Agent, which retrieves relevant insurance information using the LLM API. The collected data is then passed to the Writer Agent through a defined hand-off mechanism, where it is processed and formatted into a structured response. The final output is returned to the user through the application interface. This architecture ensures modularity, scalability, and efficient coordination between agents while leveraging the capabilities of large language models for intelligent processing.



2. SYSTEM DESIGN

2.1 Application Design

The application follows a **multi-agent architecture**, where different agents perform specialized tasks within the system. Each agent operates independently but collaborates through a structured workflow.

The **Researcher Agent** focuses on information discovery and knowledge extraction related to insurance policies. The **Writer Agent** focuses on content generation and formatting. The **Agent Orchestrator**, implemented using LangChain or CrewAI, coordinates the execution of tasks and manages communication between agents.

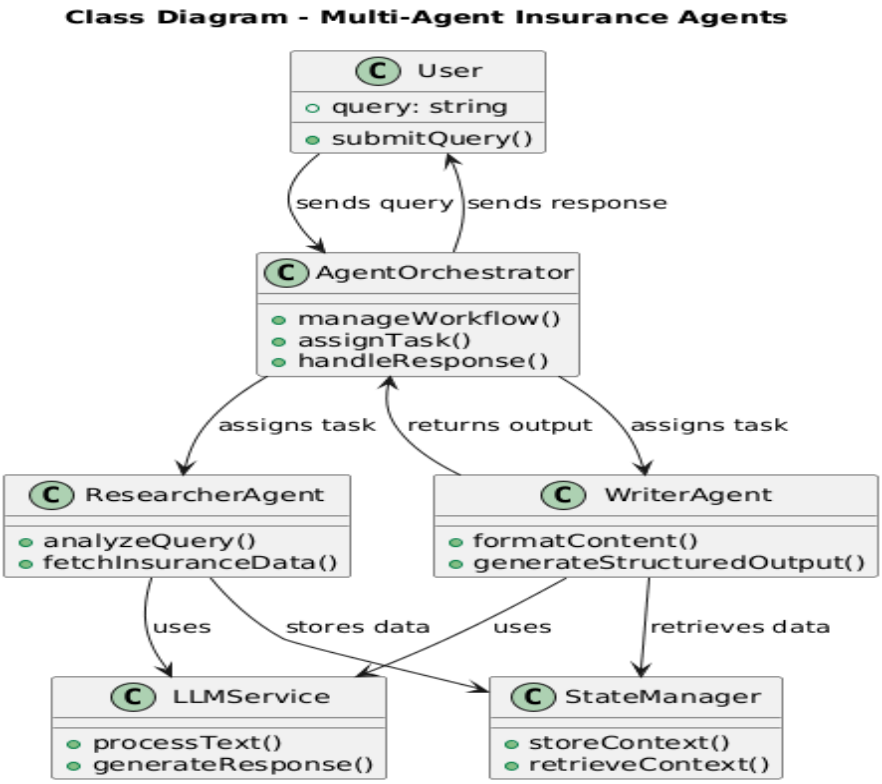
The use of modular components ensures flexibility and allows new agents to be added in the future for additional tasks such as data validation or risk analysis.

2.1.1 Class Diagram

The class diagram represents the structural design of the collaborative multi-agent system. It defines key classes such as User, Agent Orchestrator, Researcher Agent, Writer Agent, and LLM Service. The Agent Orchestrator acts as the central controller that manages task distribution and workflow execution.

The Researcher Agent is responsible for analyzing user queries and retrieving relevant insurance data, while the Writer Agent formats and generates structured outputs.

The LLM Service provides natural language processing capabilities used by both agents. This design ensures modularity, reusability, and clear separation of responsibilities, making the system scalable and easy to maintain.

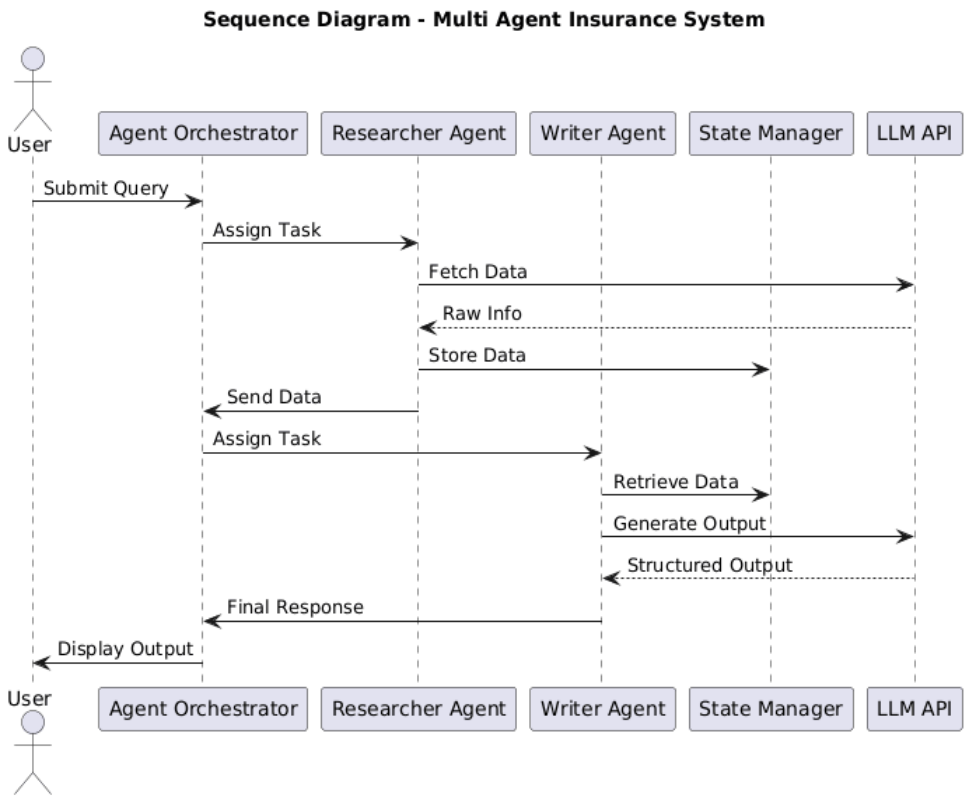


2.1.2 Sequence Diagram

The sequence diagram illustrates the dynamic interaction between system components over time. When a user submits a query, the Agent Orchestrator receives and processes it.

The Researcher Agent is first invoked to retrieve relevant insurance information using the LLM API. The intermediate data is then passed to the Writer Agent through the orchestrator. The Writer Agent formats the content into structured output using the LLM API.

Finally, the processed response is returned to the user. This diagram highlights inter-agent communication, task sequencing, and dependency on the LLM service.

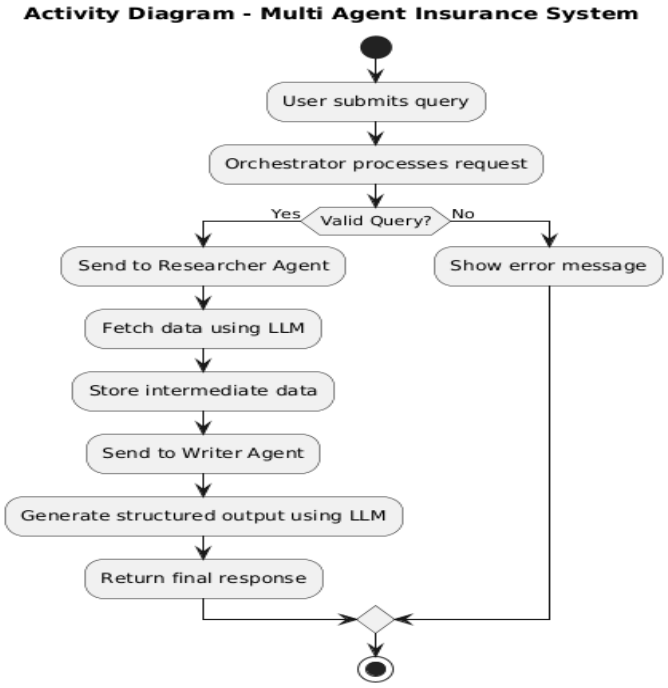


2.1.3 Activity Diagram

The activity diagram represents the workflow of the system from start to end. It begins with user input and proceeds through multiple stages including query processing, data retrieval, and content generation.

The diagram highlights the sequential execution of tasks and the collaboration between Researcher and Writer agents. Decision points can be included for handling errors or invalid inputs.

This diagram helps in understanding the overall process flow and system behavior in a simplified visual manner.



2.2 Process Flow

The process begins when the user submits an insurance-related query to the system. The query is sent to the **Researcher Agent**, which analyzes the request and retrieves relevant information using the LLM API.

Once the required data is gathered, the Researcher Agent sends the intermediate output to the **Writer Agent** through a defined hand-off protocol. The Writer Agent processes the received information and generates a structured response, including organized explanations or summaries. Finally, the system returns the formatted output to the user.

2.3 Information Flow

The information flow within the system follows a sequential pipeline. User input is first captured by the application interface and forwarded to the Researcher Agent. The Researcher Agent processes the query and generates structured intermediate information.

This intermediate data is passed to the Writer Agent, which refines and organizes the information into a clear and structured output. The final output is then delivered back to the user. The system maintains context between agents using shared state management techniques to ensure consistency and continuity.

2.4 Components Design

The system is composed of several key components that work together to implement the collaborative workflow.

The **User Interface component** allows users to submit queries and view results. The **Agent Orchestrator** coordinates the execution of tasks and manages communication between agents. The **Researcher Agent** retrieves and analyzes relevant insurance data. The **Writer Agent** converts the information into structured and readable content.

Additionally, the **LLM API component** provides natural language understanding and generation capabilities, enabling the system to process complex queries and produce meaningful responses.

2.5 Key Design Considerations

Several important design considerations were taken into account while developing the system.

First, **modularity** ensures that each agent performs a specific task independently. This simplifies development and allows future extensions. Second, **efficient communication protocols** are used to enable smooth interaction between agents. Third, **state management mechanisms** maintain context across different stages of the workflow.

The system also focuses on **scalability and flexibility**, allowing additional agents to be introduced in the future for tasks such as validation, summarization, or visualization.

2.6 API Catalogue

The Multi Agent Insurance System integrates external LLM APIs and internal agent-based APIs to perform its core functionalities. The external APIs provide natural language processing capabilities, while the internal APIs manage agent communication, task orchestration, and workflow execution between the Researcher and Writer agents.

2.6.1 LLM API (Text-to-Text Generation)

The system uses an LLM API to generate and process insurance-related information through natural language understanding and generation.

- **Service:** LLM API (e.g., OpenAI API)
- **Model:** Large Language Model (e.g., GPT-based model)
- **Purpose:** Generate and process insurance-related content and structured responses
- **Module Used:** agent_module.py → process_query()

Input

- User query (e.g., "Explain health insurance coverage")

Processing

- Query is enhanced with contextual instructions
- Sent to LLM via API request
- Used by Researcher Agent for data retrieval
- Used by Writer Agent for content generation

Output

- Structured text response:
  - Overview
  - Key Points
  - Benefits
  - Summary

2.6.2 Agent Communication API (Inter-Agent Interaction)

The system uses an internal communication mechanism to enable collaboration between agents.

- **Service:** Agent Communication Interface
- **Purpose:** Enable data transfer and coordination between Researcher Agent and Writer Agent
- **Module Used:** orchestrator.py → manage_workflow()

Input

- Intermediate data generated by Researcher Agent

Processing

- Data is passed through orchestrator
- Stored in shared state (State Manager)
- Forwarded to Writer Agent

Output

- Structured input for Writer Agent processing

2.6.3 Internal Application APIs (Agent Workflow Routes)

The application defines internal APIs to handle user queries and manage agent workflows.

Query Processing API

Agent Workflow APIs

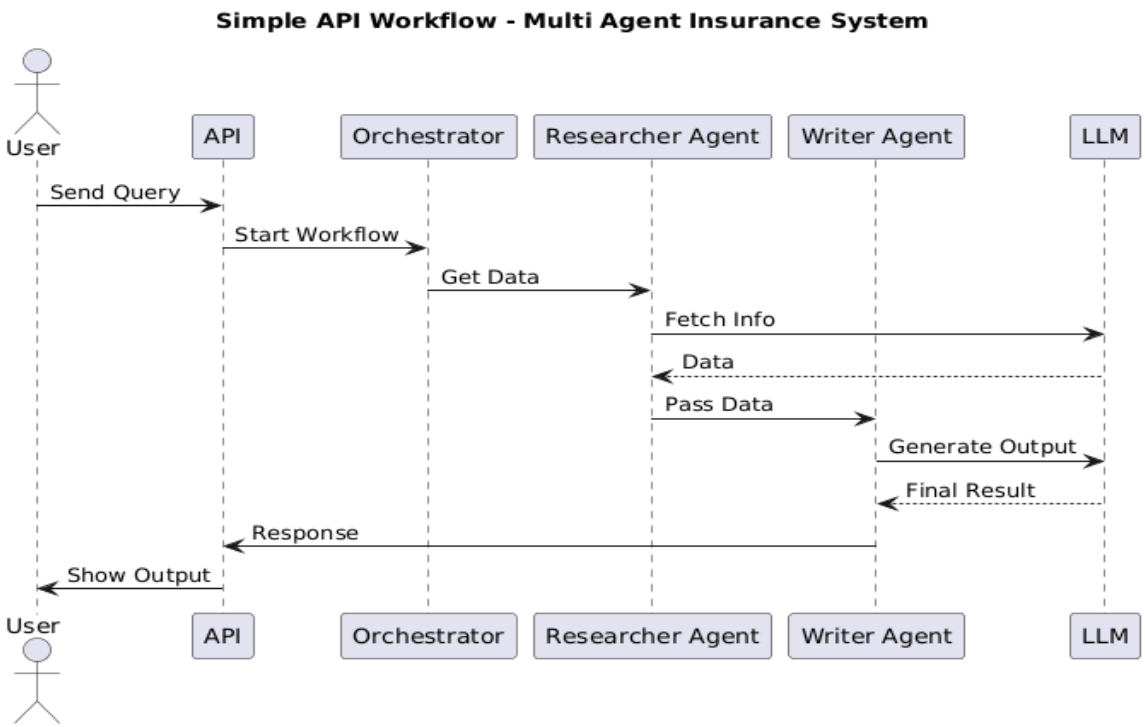
Endpoint	Method	Description
/research	POST	Triggers Researcher Agent to fetch data
/write	POST	Triggers Writer Agent to generate structured output

System Response API

Endpoint	Method	Description
/get_response	GET	Returns final structured response to user

2.6.4 API Interaction Flow

The API interaction begins when the user submits a query through the system interface. The query is sent to the internal API, which triggers the Agent Orchestrator. The orchestrator forwards the request to the Researcher Agent, which uses the LLM API to retrieve relevant insurance information. The intermediate data is then passed through the Agent Communication API to the Writer Agent. The Writer Agent processes the data using the LLM API and generates a structured response. Finally, the system returns the formatted output to the user.



REFERENCES

1. Python (programming language)

Use: Core programming language used to implement the multi-agent system, including agent logic, orchestration, and API integration.

Link:
https://docs.python.org/3/

2. LangChain

Use: Used for building and orchestrating multi-agent workflows, managing prompts, chains, and interactions between agents.

Link:
https://python.langchain.com/docs/

3. CrewAI

Use: Used for defining agent roles, collaboration, and task delegation between Researcher and Writer agents.

Link:
https://docs.crewai.com/

4. OpenAI API

Use: Provides LLM capabilities for natural language understanding and generation used by both agents.

Link:
https://platform.openai.com/docs/

5. Multi-Agent Systems

Use: Conceptual foundation for designing systems with multiple intelligent agents collaborating to achieve tasks.

Link:
https://ieeexplore.ieee.org/document/
(Search: Multi-Agent Systems overview)